

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas



Manual Técnico

GoLite - Proyecto Fase 1

Nombre Juan Francisco Ramirez de Paz

Fecha 12 de junio de 2026

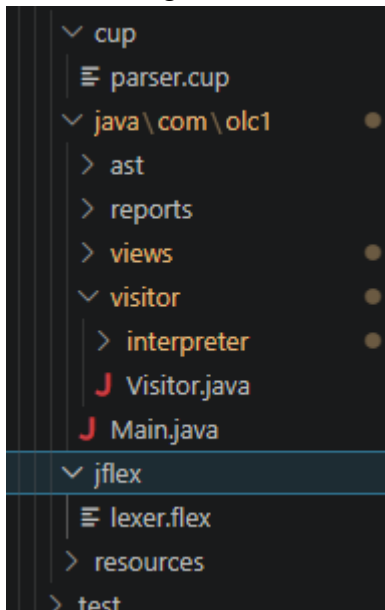
Introducción

El presente manual técnico describe la estructura interna del proyecto GoLite, así como las herramientas utilizadas para su desarrollo y el funcionamiento general del analizador léxico, sintáctico, semántico e intérprete.

El proyecto fue desarrollado en Java, utilizando JFlex para el análisis léxico, CUP para el análisis sintáctico y Java Swing para la interfaz gráfica.

Tecnologías utilizadas

- Java Maven
- JFlex
- CUP
- Java Swing



El proyecto GoLite está organizado bajo una arquitectura basada en fases de interpretación. Cada fase recibe información de la anterior y produce una salida que será utilizada por la siguiente etapa.

El flujo general del sistema es el siguiente:

Código fuente .glt



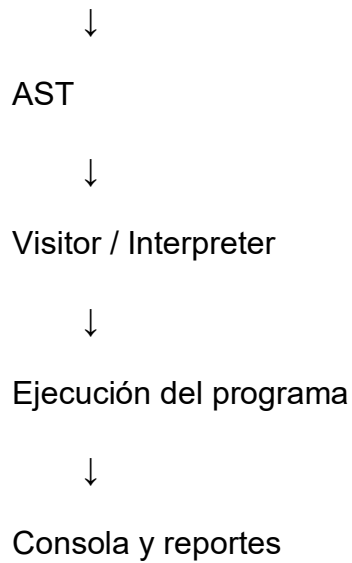
Analizador léxico JFlex



Tokens



Analizador sintáctico CUP



Esta separación permite que el proyecto sea más ordenado, ya que cada parte tiene una responsabilidad específica. El lexer se encarga de reconocer lexemas, el parser valida la estructura del lenguaje, el AST representa el programa de forma interna y el intérprete ejecuta cada instrucción.

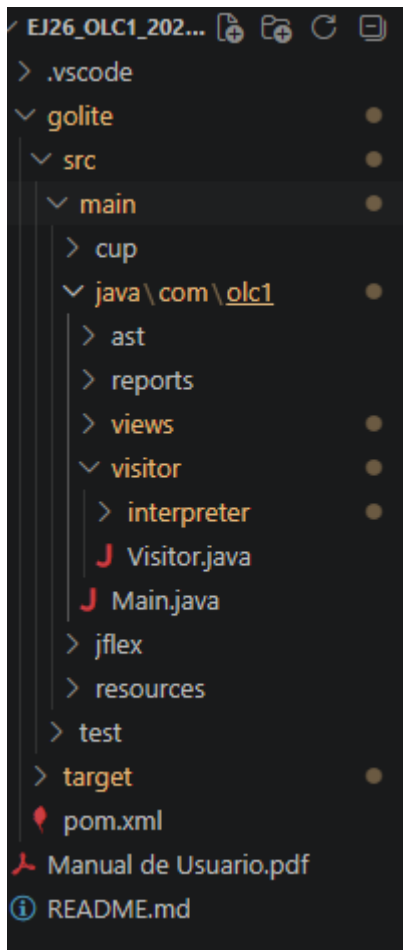
`src/main/jflex/lexer.flex`

Este archivo reconoce los lexemas del lenguaje GoLite y los convierte en tokens que luego son enviados al parser.

Entre los tokens reconocidos se encuentran:

Identificadores Enteros Decimales Cadenas Rune Palabras reservadas Operadores aritméticos Operadores lógicos Operadores relacionales Símbolos de agrupación Funciones embebidas

Además, el lexer registra errores léxicos y almacena la tabla de tokens para el reporte correspondiente.



Análisis sintáctico

El análisis sintáctico se realiza con CUP mediante el archivo:

src/main/cup/parser.cup

El parser recibe los tokens generados por el lexer y valida que cumplan con la gramática del lenguaje GoLite.

Cuando la entrada es correcta, se construye un Árbol de Sintaxis Abstracta o AST.

El parser reconoce instrucciones como:

Declaración de variables Asignaciones Impresiones If / Else if / Else For Break
Continue Bloques de sentencias 6. Árbol de Sintaxis Abstracta

El AST está formado por clases Java ubicadas en:

src/main/java/com/olc1/ast

Estas clases representan las instrucciones y expresiones del lenguaje.

Ejemplos de nodos del AST:

Add Sub Mul Div Mod VarDecl Assign AllLocate IfNode WhileFor Block Imprimir EmbeddedFunction

Cada nodo implementa la interfaz base del AST y utiliza el patrón Visitor para ser interpretado.

```
FOR_INIT ::= id:n assign EXPRESSION:e          {: RESULT = new Assign(n, e, nleft, nright); :}  
| id:i allocate EXPRESSION:e                  {: RESULT = new Allocate(i, e, ileft, iright); :}  
;  
  
FOR_UPDATE ::= id:i allocate EXPRESSION:e      {: RESULT = new Allocate(i, e, ileft, iright); :}  
| id:i plusAssign EXPRESSION:e                {: RESULT = new Allocate(i, new Add(new VarRef(i, ileft), new VarRef(i, iright))); :}  
| id:i minusAssign EXPRESSION:e               {: RESULT = new Allocate(i, new Sub(new VarRef(i, ileft), new VarRef(i, iright))); :}  
| id:i increment                             {: RESULT = new Allocate(i, new Add(new VarRef(i, ileft), new Literal(1))); :}  
| id:i decrement                             {: RESULT = new Allocate(i, new Sub(new VarRef(i, ileft), new Literal(1))); :}  
;  
  
INSTRUCCION ::= imprimir lparen rpren scol      {: RESULT = new Imprimir(new java.util.ArrayList<ASTNODE>()); :}  
| fmtPrintln lparen rpren scol                {: RESULT = new Imprimir(new java.util.ArrayList<ASTNODE>()); :}  
| imprimir lparen EXPRESIONES:e rpren scol    {: RESULT = new Imprimir(e); :}  
| fmtPrintln lparen EXPRESIONES:e rpren scol  {: RESULT = new Imprimir(e); :}  
| kWVar id:n type:t allocate EXPRESSION:e scol {: RESULT = new VarDecl(n, t, e, nleft, nright); :}  
| kWVar id:n type:t scol                      {: RESULT = new VarDecl(n, t, null, nleft, nright); :}  
| id:n assign EXPRESSION:e scol              {: RESULT = new Assign(n, e, nleft, nright); :}  
| id:i allocate EXPRESSION:e scol
```

```
ELIF ::= ELIF:el kWElse kWIf EXPRESSION:e lbrace INSTRUCCIONES:i rbrace {: el.add(new ElifNode(e, i)); RESULT = el; :}  
| kWElse kWIf EXPRESSION:e lbrace INSTRUCCIONES:i rbrace              {: RESULT = new ElifNodes(new ElifNode(e, i)); :}  
| ELIF:el kWElse:els lbrace INSTRUCCIONES:i rbrace                   {: el.add(new ElifNode(new BoolLiteral(true, elsleft, elsright), i)); :}  
| kWElse:els lbrace INSTRUCCIONES:i rbrace                           {: RESULT = new ElifNodes(new ElifNode(new BoolLiteral(true, elsleft, elsright), i)); :}  
;  
  
EXPRESIONES ::= EXPRESIONES:l comma EXPRESSION:e  {: l.add(e); RESULT = l; :}  
| EXPRESSION:e {: java.util.List<ASTNODE> list = new java.util.ArrayList<>(); list.add(e); RESULT = list; :}  
;  
  
EXPRESION ::= minus EXPRESSION:a                {: RESULT = new Negate(a); :}%prec UMENOS  
| EXPRESSION:a plus EXPRESSION:b                {: RESULT = new Add(a, b); :}  
| EXPRESSION:a minus EXPRESSION:b               {: RESULT = new Sub(a, b); :}  
| EXPRESSION:a times EXPRESSION:b               {: RESULT = new Mul(a, b); :}  
| EXPRESSION:a slash EXPRESSION:b              {: RESULT = new Div(a, b); :}  
| EXPRESSION:a mod EXPRESSION:b                 {: RESULT = new Mod(a, b); :}%prec mod  
| EXPRESSION:a equal EXPRESSION:b               {: RESULT = new Compare(a, b, "=="); :}  
| EXPRESSION:a notEqual EXPRESSION:b            {: RESULT = new Compare(a, b, "!="); :}  
| EXPRESSION:a lowertag EXPRESSION:b            {: RESULT = new LowerTag(a, b); :}  
| EXPRESSION:a greatertag EXPRESSION:b          {: RESULT = new Compare(a, b, ">"); :}  
| EXPRESSION:a lowerEqual EXPRESSION:b          {: RESULT = new Compare(a, b, "<="); :}  
| EXPRESSION:a greaterEqual EXPRESSION:b        {: RESULT = new Compare(a, b, ">="); :}  
| EXPRESSION:a and EXPRESSION:b                {: RESULT = new Logical(a, b, "&&"); :}  
| EXPRESSION:a or EXPRESSION:b                 {: RESULT = new Logical(a, b, "||"); :}  
| not EXPRESSION:e                             {: RESULT = new Logical(e, null, "!"); :}%prec not
```

Intérprete

La interpretación del AST se realiza mediante la clase:

src/main/java/com/olc1/visitor/interpreter/InterpreterVisitor.java

Esta clase recorre los nodos del AST y ejecuta las instrucciones del programa.

Entre sus responsabilidades están:

Evaluar expresiones Ejecutar instrucciones Controlar ámbitos Validar tipos Manejar errores semánticos Controlar sentencias break y continue Ejecutar funciones embebidas 8. Manejo de ámbitos

El manejo de ámbitos se realiza mediante la clase Enviroment.

Cada bloque de sentencias puede crear un nuevo entorno. Esto permite manejar correctamente variables locales, shadowing y visibilidad de variables dentro y fuera de bloques.

```
package com.olc1.visitor.interpreter;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import com.olc1.ast.ASTNODE;
import com.olc1.ast.exp.*;
import com.olc1.ast.stm.*;
import com.olc1.reports.GoliteError;
import com.olc1.visitor.Visitor;
import com.olc1.visitor.interpreter.value.*;
import com.olc1.visitor.interpreter.transfer.BreakException;
import com.olc1.visitor.interpreter.transfer.ContinueException;

public class InterpreterVisitor implements Visitor<ValueWrapper> {
    public String output = "";
    private final ValueWrapper defaultVoid = new VoidValue(-1, -1);
    private Enviroment enviroment = new Enviroment();
    private int loopDepth = 0;
    public final List<GoliteError> errors = new ArrayList<>();

    private void executeInNewScope(ASTNODE body) {
        Enviroment previous = this.enviroment;

        try {
            this.enviroment = new Enviroment(previous);
            Visit(body);
        } finally {
            this.enviroment = previous;
        }
    }
}
```

Tipos de datos

El lenguaje implementa tipos primitivos mediante clases de valor.

Entre los tipos soportados se encuentran:

int float64 string bool rune nil

Cada tipo se representa mediante una clase que implementa la interfaz ValueWrapper.

```
package com.olc1.visitor.interpreter.value;

public sealed interface ValueWrapper
    permits IntValue, DecimalValue, VoidValue, BoolValue, StringValue, NilValue, RuneValue {

    int line();
    int column();
    String getTypeName();
}
```

Manejo de errores

El proyecto maneja tres tipos de errores:

Errores léxicos Errores sintácticos Errores semánticos

Los errores se almacenan en objetos de la clase:

com.olc1.reports.GoliteError

Cada error contiene:

Tipo Descripción Línea Columna

Estos errores se muestran en una ventana de reporte dentro de la interfaz gráfica.


```

package com.olc1.reports;

✓ public class GoliteError {
    private final String type;
    private final String description;
    private final int line;
    private final int column;

    ✓ public GoliteError(String type, String description, int line, int column) {
        this.type = type;
        this.description = description;
        this.line = line;
        this.column = column;
    }

    ✓ public String getType() {
        return type;
    }

    ✓ public String getDescription() {
        return description;
    }

    ✓ public int getLine() {
        return line;
    }

    ✓ public int getColumn() {
        return column;
    }
}

```

Reporte de tokens

El reporte de tokens se maneja mediante la clase:

com.olc1.reports.TokenReport

Cada token registrado contiene:

Lexema Tipo de token Línea Columna

La información se muestra en una tabla dentro de una ventana de reportes.

```

package com.olc1.reports;

public class TokenReport {
    private final String lexeme;
    private final String tokenType;
    private final int line;
    private final int column;

    public TokenReport(String lexeme, String tokenType, int line, int column) {
        this.lexeme = lexeme;
        this.tokenType = tokenType;
        this.line = line;
        this.column = column;
    }

    public String getLexeme() {
        return lexeme;
    }

    public String getTokenType() {
        return tokenType;
    }

    public int getLine() {
        return line;
    }

    public int getColumn() {
        return column;
    }
}

```

Gramática BNF resumida

::=

::= |

::= "fmt.Println" "(" ")" | "print" "(" ")" | "var" id "=" | "var" id | id "!=" | id "=" | id "+=" | id
 "-=" | "if" "{" "}" | "if" "{" "}" | "for" "{" "}" | "for" ";" ";" "{" "}" | "break" | "continue" | "{" "}"

::= "else" "if" "{" "}" | "else" "{" "}" | ε

::= "int" | "float64" | "string" | "bool" | "rune"

::= id "!=" | id "="

::= id "=" | id "+=" | id "-=" | id "++" | id "--"

::= "," |

```
::= "+" | "-" | "*" | "/" | "%" | "==" | "!=" | "<" | ">" | "<=" | ">=" | "&&" | "||" | "!" | "-" | "(" ")"  
| id | entero | decimal | cadena | rune | "true" | "false" | "nil" | "strconv.Atoi" "(" ")" |  
"strconv.ParseFloat" "(" ")" | "reflect.TypeOf" "(" ")" ".string" "(" ")"
```

::= ";" | salto_linea 13. Compilación del proyecto

Para compilar el proyecto se utiliza Maven:

```
mvn clean compile
```

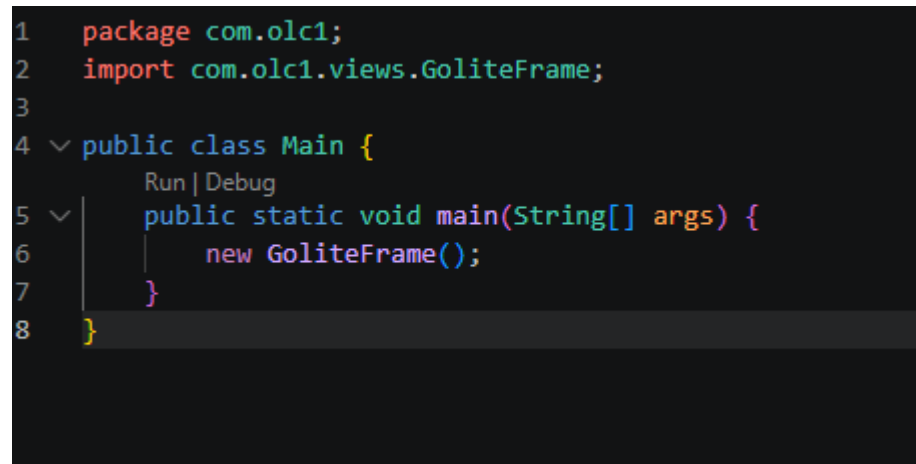
Este comando genera los analizadores y compila el proyecto completo.

Ejecución del proyecto

La ejecución se realiza desde la clase principal:

```
src/main/java/com/olc1/Main.java
```

Al ejecutar esta clase se inicia la interfaz gráfica del intérprete.



```
1 package com.olc1;  
2 import com.olc1.views.GoliteFrame;  
3  
4 public class Main {  
5     Run | Debug  
6     public static void main(String[] args) {  
7         new GoliteFrame();  
8     }  
}
```

Patrón Visitor

El proyecto utiliza el patrón de diseño Visitor para recorrer e interpretar los nodos del AST. Cada nodo del árbol implementa un método `accept`, el cual recibe un visitor y le delega la ejecución correspondiente. Este patrón permite separar la estructura del AST de la lógica de ejecución. De esta forma, los nodos solamente representan instrucciones o expresiones, mientras que la clase `InterpreterVisitor` se encarga de interpretar su comportamiento.

Ejemplo general del flujo:

Nodo AST

↓

`accept(visitor)`

↓

`visit(nodo)`

↓

ejecución semántica

El uso del patrón Visitor facilita agregar nuevas instrucciones o expresiones al lenguaje sin modificar completamente la estructura existente.

```
import com.olc1.visitor.interpreter.value.*;
import com.olc1.visitor.interpreter.transfer.BreakException;
import com.olc1.visitor.interpreter.transfer.ContinueException;
import com.olc1.visitor.interpreter.transfer.ReturnException;
import com.olc1.visitor.interpreter.transfer.BreakException;
import com.olc1.visitor.interpreter.transfer.ContinueException;

public class InterpreterVisitor implements Visitor<ValueWrapper> {
    public String output = "";
    private final ValueWrapper defaultVoid = new VoidValue(-1, -1);
    private Environment environment = new Environment();
    private int loopDepth = 0;
    private int switchDepth = 0;
    private final Map<String, FunctionDecl.Context> functions = new HashMap<>();
    private final Map<String, StructTypeDecl.Context> structTypes = new HashMap<>();
    private int statementsDepth = 0;
    private final Map<String, MethodDecl.Context> methods = new HashMap<>();
    public final List<GoliteError> errors = new ArrayList<>();
    public final List<SymbolReport> symbols = new ArrayList<>();
    private final Set<String> symbolKeys = new HashSet<>();
    private String currentScope = "Global";

    private void executeInNewScope(ASTNODE body) {
```

Manejo de entornos

El manejo de ámbitos se realiza mediante la clase Enviroment. Cada entorno almacena variables declaradas y puede tener una referencia a un entorno padre.

Esto permite representar correctamente los ámbitos anidados del lenguaje, como bloques, funciones, ciclos y métodos.

Cuando se busca una variable, primero se consulta el entorno actual. Si no existe, se busca en el entorno padre. Si no se encuentra en ningún nivel, se genera un error semántico.

Este mecanismo permite manejar:

Variables locales

Variables globales

Shadowing

Parámetros de funciones

Variables dentro de bloques

Variables dentro de ciclos

Receivers de métodos

```
1  package com.olc1.visitor.interpreter;
2
3  import java.util.HashMap;
4  import java.util.Map;
5
6  import javax.management.RuntimeErrorException;
7  import com.olc1.visitor.interpreter.value.DecimalValue;
8  import com.olc1.visitor.interpreter.value.IntValue;
9  import com.olc1.visitor.interpreter.value.ValueWrapper;
10
11
12  public class Enviroment {
13      private final Map<String, ValueWrapper> variables;
14      private final Enviroment parent;
15
16      public Enviroment() {
17          variables = new HashMap<>();
18          this.parent = null;
19      }
20
21      public Enviroment(Enviroment parent) {
22          variables = new HashMap<>();
23          this.parent = parent;
24      }
25
26      public ValueWrapper declare(String name, ValueWrapper value) {
27          if (variables.containsKey(name)) {
28              throw new RuntimeException("Variable '" + name + "' ya declarada en este ámbito");
29          }
30          variables.put(name, value);
31          return value;
32      }
33  }
```

Métodos de structs

GoLite también permite declarar métodos asociados a structs.

Ejemplo:

```
type Persona struct {  
    nombre string  
    edad int  
}  
  
func (p Persona) saludar() {  
    fmt.Println("Hola", p.nombre)  
}
```

Los métodos se registran utilizando una llave compuesta por el nombre del struct y el nombre del método. Esto permite diferenciar métodos pertenecientes a distintos structs.

El intérprete valida:

- Que el struct exista
- Que el método no esté duplicado
- Que los parámetros sean correctos
- Que el receiver sea válido
- Que los retornos coincidan con el tipo declarado

Structs

Los structs permiten agrupar varios atributos bajo un mismo tipo definido por el usuario.

Ejemplo:

```
type Persona struct {  
    nombre string  
    edad int  
}
```

El intérprete permite crear valores de tipo struct, acceder a sus atributos y modificar sus valores.

Ejemplo:

```
persona := Persona{nombre: "German", edad: 22}  
fmt.Println(persona.nombre)  
persona.edad = 23
```

Se validan errores como:

Struct no declarado

Struct vacío

Atributo inexistente

Atributo repetido

Tipo incorrecto en asignación de atributo

Inicialización repetida de atributos

```
package com.olc1.visitor.interpreter.value;

import java.util.LinkedHashMap;
import java.util.Map;

public record StructValue(String structName, Map<String, ValueWrapper> fields, int line, int column) implements Struct {

    @Override
    public String getTypeName() {
        return structName;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append(structName).append(str: "{");

        int i = 0;

        for (Map.Entry<String, ValueWrapper> entry : fields.entrySet()) {
            if (i > 0) {
                sb.append(str: ", ");
            }

            sb.append(entry.getKey()).append(str: ": ").append(entry.getValue().toString());
            i++;
        }

        sb.append(str: "}");
        return sb.toString();
    }
}
```

Slices

Los slices permiten almacenar colecciones de valores de un mismo tipo.

Ejemplo:

```
numeros := []int{10, 20, 30}
```

```
fmt.Println(numeros[0])
```

```
numeros[1] = 99
```

El lenguaje soporta:

Declaración de slices

Acceso por índice
Asignación por índice
Slices vacíos
Slices multidimensionales
append
len
slices.Index
strings.Join
for range

```
package com.olc1.visitor.interpreter.value;

import java.util.List;

public record SliceValue(String elementType, List<ValueWrapper> values, int line, int column) implements ValueWrapper {

    @Override
    public String getTypeName() {
        return "[" + elementType;
    }

    @Override
    public String toString() {
        return values.toString();
    }
}
```

El intérprete valida:

- Que el índice sea entero
- Que el índice esté dentro del rango válido
- Que los valores asignados sean compatibles con el tipo del slice
- Que append reciba un valor compatible
- Que strings.Join reciba un []string
- Que slices.Index busque un valor compatible

Reporte de tabla de símbolos

La tabla de símbolos almacena información de los identificadores encontrados durante la interpretación del programa.

Cada símbolo contiene:

- ID
- Tipo de símbolo
- Tipo de dato
- Ámbito
- Línea
- Columna

Los tipos de símbolos registrados pueden ser:

- Variable
- Función
- Procedimiento
- Parámetro
- Struct
- Atributo
- Método
- Receiver

Este reporte permite visualizar de forma ordenada los elementos declarados dentro del programa y el ámbito en el que pertenecen.

Reporte AST

El reporte AST permite visualizar gráficamente la estructura del programa interpretado.

Para generar este reporte se utiliza Graphviz. El proyecto genera un archivo DOT de forma interna y luego lo convierte en imagen usando el ejecutable dot.

El AST se presenta como un árbol donde se muestran instrucciones, expresiones, operadores, llamadas a funciones, argumentos, estructuras de control y demás nodos que forman parte del programa.

Este reporte facilita la comprensión de cómo el parser construye internamente la representación del código fuente.

Conclusión

El proyecto GoLite implementa las fases principales de un intérprete básico: análisis léxico, análisis sintáctico, análisis semántico y ejecución del AST. Además, incluye una interfaz gráfica y reportes que permiten visualizar errores y tokens reconocidos durante el análisis.